

How the Parameters Get Inferred

A sequential call trace of your script — which `.py` file and which **function** runs at each step.
Configuration: **viscoelastic + bounded**, $\theta = 45^\circ$, `use_y=True`, `sigma_bias=None`.

What your settings resolve to

<code>self.model</code>	<code>viscoelastic_bounded</code> (<code>theta != 90</code> , so not the <code>_perp</code> variant)
Fitted components	<code>x</code> and <code>y</code> (<code>use_y=True</code> + <code>viscoelastic</code> + <code>bounded</code>)
Inferred parameters	<code>eta_s</code> , <code>eta_p</code> , <code>lambda_</code> , <code>delta0</code> (sampled in <code>log10</code>) + <code>sigma_noise</code> (physical)
Not inferred	<code>sigma_bias</code> (= <code>None</code> -> discrepancy term off, likelihood is diagonal)
<code>ndim</code> / <code>nwalkers</code>	5 / 10 (<code>nwalkers</code> = 2 x <code>ndim</code>)
<code>theta_true</code>	[0.5, 0.9, 0.1, 0.1] = (<code>eta_s</code> , <code>eta_p</code> , <code>lambda_</code> , <code>delta0</code>)

All methods below live on the single `InferenceProcedure` object, built from mixins: `WallCorrections`, `ForwardModels`, `DataIO`, `Transforms`, `Priors`, `Likelihood`, `Sampler`, `Plotting` (see `main.py`).

Step 0 — construct the model

`model = InferenceProcedure(...)`

Records the settings, decides which model string and which parameters are being fitted, and sizes the sampler.

File	Function	What it does
<code>main.py</code>	<code>InferenceProcedure.__init__</code>	Stores <code>force</code> , <code>theta</code> , <code>delta0</code> , <code>t_unload</code> , <code>bounds</code> , <code>theta_true</code> ; builds <code>self.model = 'viscoelastic_bounded'</code> .
<code>main.py</code>	<code>_get_ndim</code>	Counts sampler dimensions: material params + <code>sigma_noise</code> (+ <code>sigma_bias</code> if inferred).
<code>main.py</code>	<code>_get_parameter_names</code>	Returns [<code>eta_s</code> , <code>eta_p</code> , <code>lambda_</code> , <code>delta0</code>] for this model.
<code>main.py</code>	<code>_bias_is_inferred</code>	False here (<code>sigma_bias=None</code>), so no bias dimension is added.

Step 1 — load the data

`model.load_data("data/linear_viscoelastic/angle45/delta-0.1-45.out")`

Reads the FOM file, selects the `x` and `y` columns, thins, and adds synthetic measurement noise. Also sets the noise prior scale from the data.

File	Function	What it does
<code>data_io.py</code>	<code>DataIO.load_data</code>	Top-level loader; checks <code>t_unload</code> is finite, reads the file, stores <code>self.data</code> .
<code>data_io.py</code>	<code>_parse_columns</code>	Picks columns; returns components ('x','y') because <code>use_y</code> is on.
<code>data_io.py</code>	<code>_add_noise</code>	Adds $N(0, 2\%$ of peak) noise per component; keeps noise-free <code>x_clean</code> / <code>y_clean</code> .
<code>data_io.py</code>	<code>_sigma_priors_from_data</code>	Sets $\beta = 1/(0.10 \times \text{peak})$ -> <code>sigma_noise_prior</code> .
<code>data_io.py</code>	<code>_report</code>	Prints the load summary you see in the console.

Step 2 — data vs. model at the truth

`model.plot_data()`

Draws observed FOM points against the SAM curve at `theta_true`. First place the forward model runs.

File	Function	What it does
plotting.py	Plotting.plot_data	Loops over components; draws FOM points and the SAM line.
plotting.py	_resolve_theta_true	Falls back to self.theta_true = [0.5, 0.9, 0.1, 0.1].
forward_models.py	_model_component	Dispatch: sends (eta_s, eta_p, lambda_) + delta0 to the viscoelastic model.
forward_models.py	model_viscoelastic	Chooses the bounded 2-D path for theta=45 with use_y.
forward_models.py	_viscoelastic_bounded_full	Builds the coupled ODE (creep + polymer drag) and integrates it.
forward_models.py	_solve_creep_recovery	Runs scipy solve_ivp over loading then recovery at t_unload.
wall_corrections.py	_wall_factors / zeng_parallel / brenner_perpendicular	Evaluate f_parallel, f_perp at gap $\delta(t) = \delta_0 + y$.

Step 3 — model discrepancy

model.plot_model_error()

Evaluates the SAM at theta_true on the data grid and compares to the noise-free FOM (x_clean / y_clean); prints max, mean, relative-L2 error per component.

File	Function	What it does
plotting.py	Plotting.plot_model_error	Computes FOM - SAM and the relative-L2 norm; plots it.
forward_models.py	_model_component	Same forward-model chain as Step 2 (on the data time grid).

Step 4 — show the priors

model.plot_prior()

Draws each marginal prior: flat/uniform on log10 for material parameters, exponential for sigma_noise.

File	Function	What it does
plotting.py	Plotting.plot_prior	Uniform density on log10 for material params; $\text{rate} \cdot \exp(-\text{rate} \cdot x)$ for sigma_noise.
main.py	_get_parameter_bounds / _get_parameter_labels	Provide the bounds and labels drawn on the axes.

Step 5 — the inference itself

`samples = model.run_mcmc(warmup=True)`

This is where the parameters are actually inferred. emcee walkers explore the posterior; every proposal is scored by the log-posterior, which runs the forward model and compares it to the data.

File	Function	What it does
sampler.py	<code>Sampler.run_mcmc</code>	Driver: builds walkers, runs warm-up, runs the main chain, summarises.
sampler.py	<code>sample_starting_points</code>	Seeds walkers by k-means clustering a prior pool (kept finite).
sampler.py	<code>_prior_draws</code>	Draws the prior pool: log10-uniform material params, exponential <code>sigma_noise</code> .
sampler.py	<code>_warmup_starting_points</code>	Short preliminary chain; restarts walkers near the best positions.
sampler.py	<code>_print_walkers</code>	Prints walker positions (via <code>_to_physical</code>).
emcee	<code>EnsembleSampler.run_mcmc</code>	Affine-Invariant sampler; calls <code>log_posterior</code> for every walker/step.

Inside every posterior evaluation (called ~nwalkers x nsteps times)

`log_posterior = log_prior + log_likelihood`. The likelihood pulls the parameters toward values that reproduce your x and y data.

File	Function	What it does
likelihood.py	<code>Likelihood.log_posterior</code>	Sums <code>log_prior</code> and <code>log_likelihood</code> ; returns -inf if either is invalid.
priors.py	<code>Priors.log_prior</code>	log10-uniform inside bounds for material params; exponential for <code>sigma_noise</code> .
priors.py	<code>_log_exponential_prior</code>	log density of the exponential noise prior.
logtransforms.py	<code>_to_physical</code> / <code>_get_log10_mask</code>	Maps sampler coords -> physical ($\theta = 10^{\phi}$ on log10 dims).
likelihood.py	<code>Likelihood.log_likelihood</code>	Builds residuals per component and scores them under the Gaussian model.
likelihood.py	<code>_residuals</code>	observed - model for x and y.
forward_models.py	<code>_model_component -> _viscoelastic_bounded_full</code>	THE forward model: solve_ivp + wall factors (as in Step 2).
main.py	<code>_extract_noise_bias</code>	Splits <code>sigma_noise</code> out of the parameter vector (no bias here).
likelihood.py	<code>_log_likelihood</code>	Diagonal Gaussian score (<code>sigma_bias=None</code> , so no covariance matrix).

After sampling

File	Function	What it does
sampler.py	<code>run_mcmc (tail)</code>	Discards <code>burn_fraction</code> as burn-in; maps chain to physical via <code>_to_physical</code> -> <code>self.samples</code> .
sampler.py	<code>print_results</code>	Posterior mean, std, credible interval, and R-hat per parameter.
sampler.py	<code>_gelman_rubin_diag</code>	Gelman-Rubin R-hat convergence statistic.

Step 6 — results figures

model.plot_results()

Draws the three result figures from the posterior samples.

File	Function	What it does
plotting.py	Plotting.plot_results	Calls the three plotters below.
plotting.py	plot_posterior_predictive / _predictive_summary	Posterior predictive band vs data; coverage diagnostics.
plotting.py	plot_corner	Corner plot of the material parameters (prior, truth, credible interval).
plotting.py	plot_trace	Per-parameter walker chains with the burn-in marked.

In one line: the parameters are inferred in `sampler.py` (emcee driving `run_mcmc`), which repeatedly evaluates `log_posterior` in `likelihood.py` = `log_prior` (`priors.py`) + `log_likelihood`; the likelihood runs the forward model in `forward_models.py` (`_viscoelastic_bounded_full` via `solve_ivp` + wall corrections) and compares it to your x and y data, with `logtransforms.py` mapping between log10 and physical units.